

SIMATS ENGINEERING ASSESSMENT TOOL 2

NAME: JAIVANT.S

REG NO: (192311326)

COURSE NAME: Design and Analysis of Algorithms

COURSE CODE: CSA0613

Scenario Based Assessment

Q13. Linear Search for Student ID Lookup

Scenario: A university portal stores student IDs in an unsorted format due to continuous registrations. During exams, quick lookup of student IDs is required.

a) How Linear Search works for retrieving student records

Linear Search (also called Sequential Search) checks each student record one by one, starting from the first entry, until it finds the target student ID or reaches the end of the list. Since the IDs are unsorted, this is the only search method that reliably works without first organizing the data.

Steps of the algorithm

- Start at the first record in the student ID list (index 0).
- Compare the target student ID with the ID at the current position.
- If it matches, return that record (search succeeds) and stop.
- If it does not match, move to the next record (index + 1).
- Repeat the comparison until a match is found or the list ends.
- If the end of the list is reached with no match, report that the student ID does not exist.

Because registrations happen continuously, new student IDs are simply appended to the end of the list. Linear Search does not require the data to be in any particular order, so it works correctly regardless of how the records were added — but it inspects records one at a time, so performance depends directly on how many records must be checked before a match is found.

Source Code:

```
1 def linear_search(student_ids, target):
2     for i in range(len(student_ids)):
3         if student_ids[i] == target:
4             return i
5     return -1
6
7
8 # Input student IDs
9 n = int(input("Enter number of student IDs: "))
10
11 student_ids = []
12
13 print("Enter Student IDs:")
14 for i in range(n):
15     student_ids.append(int(input()))
16
17 target = int(input("Enter Student ID to search: "))
18
19 result = linear_search(student_ids, target)
20
21 if result != -1:
22     print("Student ID found at position:", result + 1)
23 else:
24     print("Student ID not found.")
```

Output:

Output

Clear

```
Enter number of student IDs: 5
Enter Student IDs:
101
205
309
412
550
Enter Student ID to search: 309
Student ID found at position: 3

=== Code Execution Successful ===
```

b) Best, average, and worst-case time complexities

Let n represent the total number of student records stored in the portal.

- Best Case — $O(1)$: The target student ID is the very first record checked. Only one comparison is needed, so the time taken does not depend on n .
- Average Case — $O(n)$: On average, the target ID is found somewhere near the middle of the list, requiring roughly $n/2$ comparisons. Since constant factors are dropped in Big-O notation, this simplifies to $O(n)$.
- Worst Case — $O(n)$: The target ID is either the last record in the list, or it does not exist in the list at all. In both situations, all n records must be examined before the search can conclude.

Space complexity is $O(1)$, since Linear Search only needs a single index/pointer to traverse the list and does not require extra memory proportional to n .

c) Would sorting the dataset improve performance?

Yes, sorting would significantly improve lookup performance, but the benefit comes with trade-offs that must be weighed against the university portal's requirements.

Why sorting helps

- Once IDs are sorted, Binary Search can be used instead of Linear Search, reducing the time complexity from $O(n)$ to $O(\log n)$ for every lookup.
- During exams, when many quick lookups happen in a short window, $O(\log n)$ search is dramatically faster than $O(n)$ search, especially as the number of enrolled students grows large.

Trade-offs to consider

- Sorting itself has a cost: a comparison-based sort takes $O(n \log n)$ time, so it is only worthwhile if many searches will be performed afterward (which is exactly the exam-lookup scenario described).
- Because registrations are continuous, new IDs keep arriving. Inserting a new ID into a sorted array costs $O(n)$ in the worst case (to maintain order), compared to $O(1)$ for simply appending to an unsorted list.
- A practical approach is to keep new registrations in an unsorted "incoming" list and periodically re-sort (or merge them into a sorted structure, or use a hash table/index) before the exam period begins, so lookups during exams get the $O(\log n)$ or even $O(1)$ benefit without paying the re-sort cost during every single registration.

Conclusion: Sorting is justified here because the read pattern (many rapid lookups during exams) heavily outweighs the one-time or periodic cost of sorting. For a system with continuous insertions but a predictable high-lookup exam period, sorting the dataset (or using a hash-based index for $O(1)$ average lookup) is the recommended optimization over plain Linear Search.

Report

Aim

To implement Linear Search for retrieving student records from an unsorted list of student IDs and analyze its performance.

Algorithm

1. Read the number of student IDs.
2. Store the IDs in a list.
3. Read the target student ID.
4. Compare each ID with the target.
5. If found, display its position.
6. Otherwise, display "Student ID not found."

Time Complexity

Best Case: $O(1)$

Average Case: $O(n)$

Worst Case: $O(n)$

Space Complexity

$O(1)$ (excluding the input list)

Conclusion

Linear Search is suitable for unsorted student records because it requires no preprocessing. However, if the dataset is searched frequently, sorting the records and using Binary Search provides much faster lookups.

Assessment Rubric Reference

The table below reproduces the grading rubric used to evaluate responses to this question.

Criteria	Weightage	Exemplary	Proficient	Developing	Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification